# Data visualisation and exploration

Thomas de Jaeger

thomas.de-jaeger@epita.fr

Spring 2026

# Visualizing Multidimensional Data & Dimensionality Reduction

Session 6

# Why High-Dimensional Visualization Matters

**Real-world data often live in high-dimensional spaces**

Examples:

- Images: MNIST digits are 28×28 pixels = 784 features (treated as a 784D vector in ML)
- Text embeddings: 300–1000 dimensions
- Genomics: 20,000+ gene expression features
- Finance: 100+ simultaneous indicators

We cannot visualize 784D or 20,000D.

Dimensionality reduction is important: we need methods to **compress high-dimensional representations into 2D or 3D** so we can explore structure, clusters, or anomalies

# Human Perception Limits

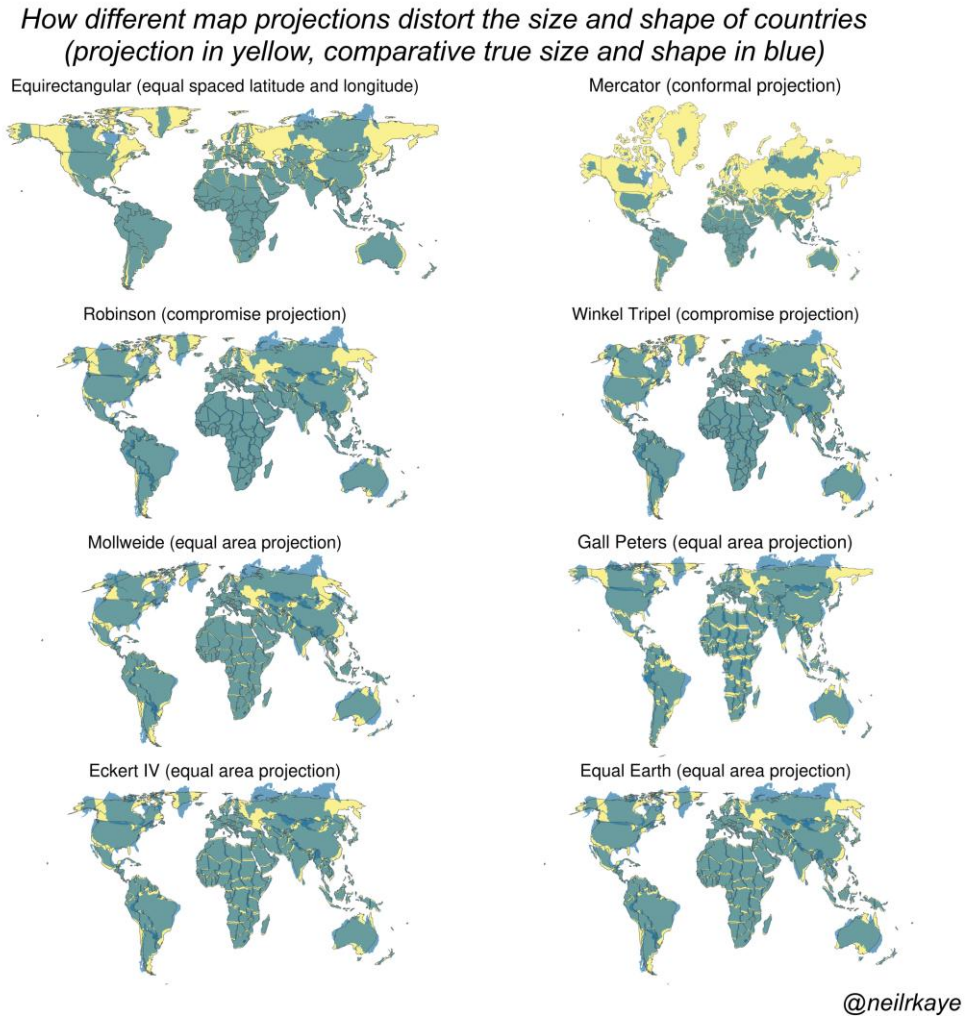
Humans perceive 2D naturally. We are very comfortable with 2D plots:
scatterplots, bar charts, histograms

3D possible, but requires rotation/interaction.

Beyond 3D → impossible to visualize directly.

Analogy: flattening a globe → 2D world map.

All methods for high-dimensional visualization rely on **projection** into 2D or 3D. But projection always comes with **distortions**. Just like when we flatten a 3D globe into a 2D map



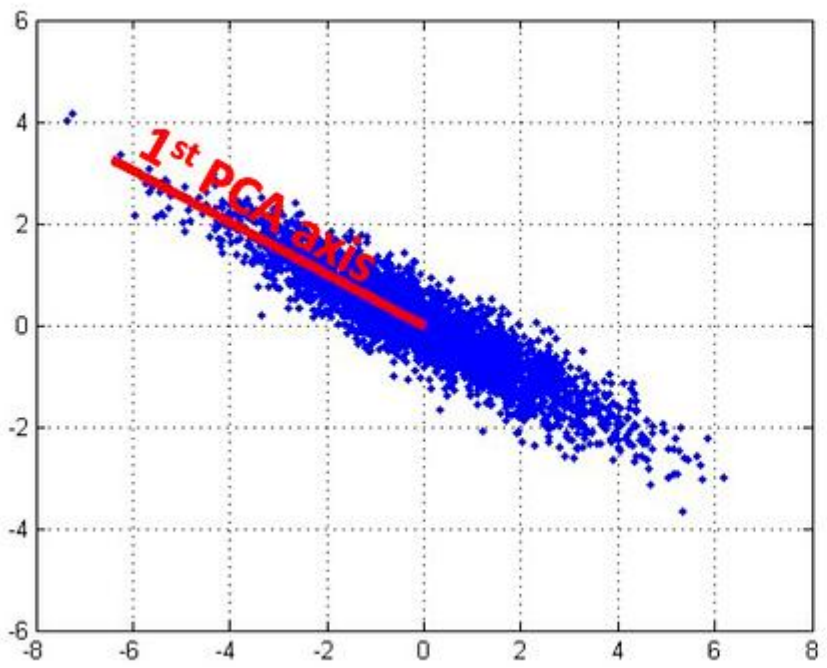
# Approaches to Visualization

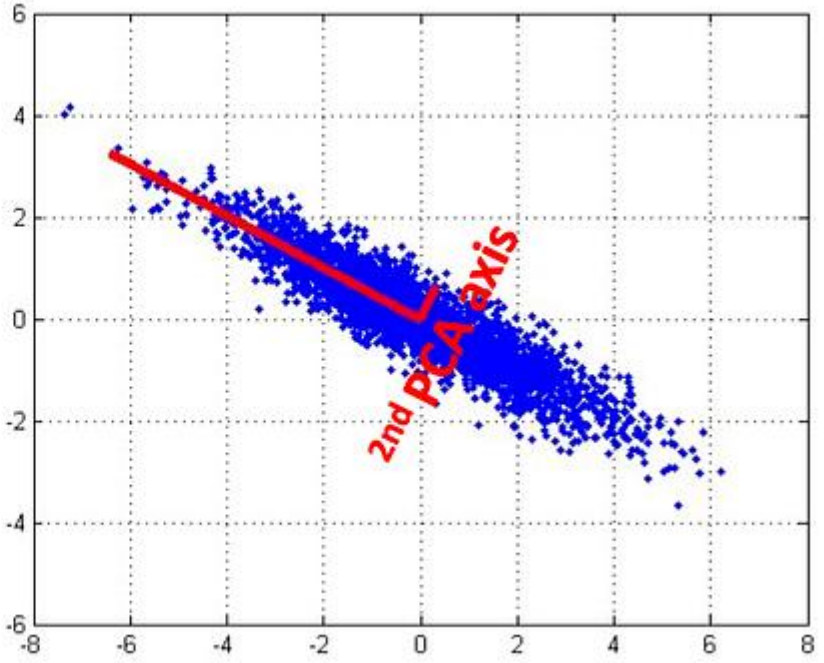
- **Feature Selection:** just keep the most important features, like picking top 3 indicators in finance.
- **Feature Extraction:** transform features into fewer, more meaningful dimensions — PCA is the most famous example.
- **Projection Methods:** explicitly map high dimensions into 2D/3D. PCA, t-SNE, and UMAP.
- **Multi-dimensional Plots:** parallel coordinates or radar plots attempt to directly visualize many dimensions without reducing them.

# Principal Component Analysis

**PCA:** is a dimensionality reduction technique that transforms high-dimensional data into a lower-dimensional form while **preserving as much variance as possible**. PCA achieves this by finding new axes, called principal components, which maximize variance in the data.

The first component captures the most variance, the second captures the next most while being orthogonal, and so on.

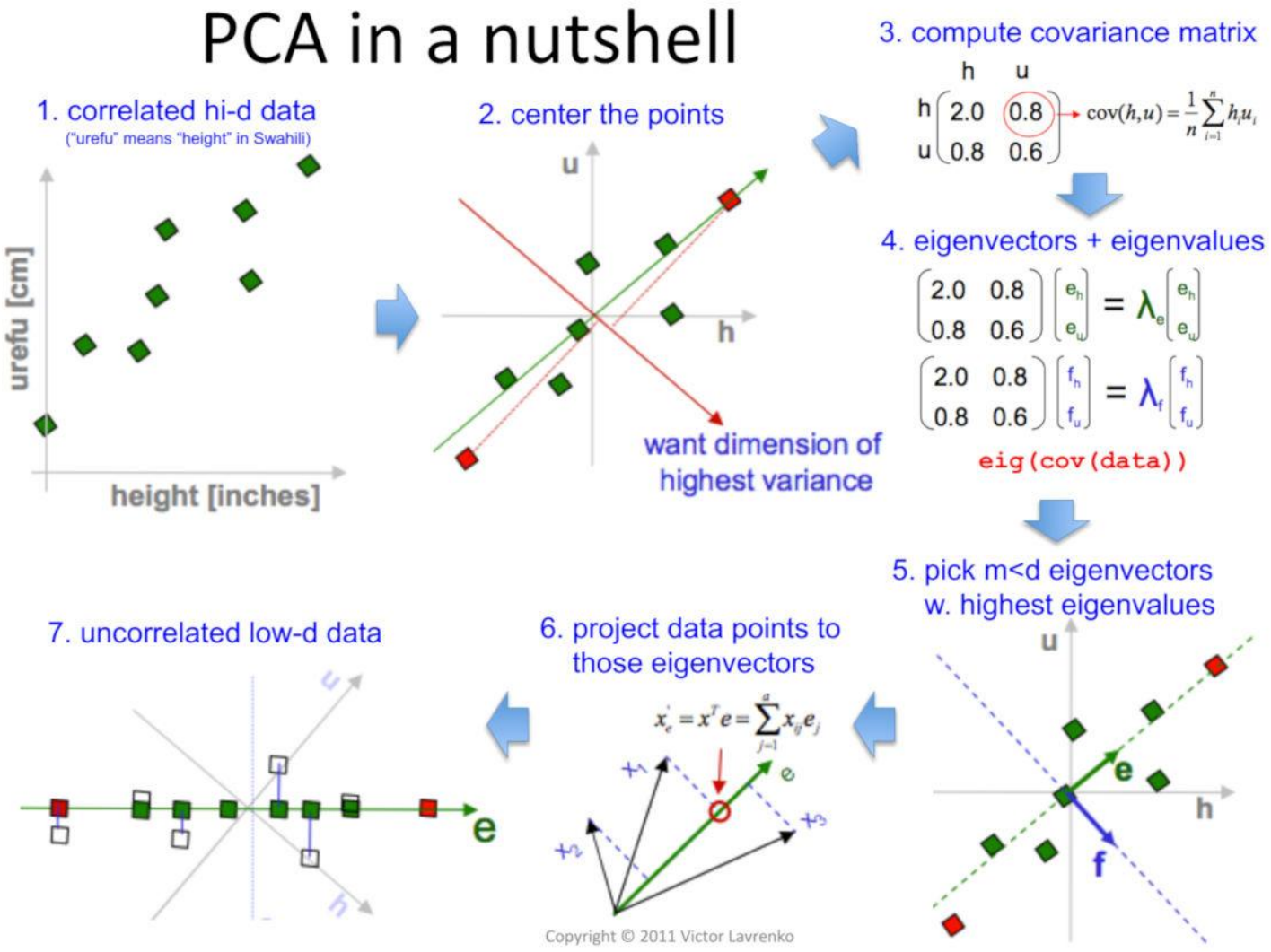


# Principal Component Analysis

**How it works?**

- **Standardize the Data:** Ensure that each feature has a mean of zero and a standard deviation of one.
- **Compute the Covariance Matrix:** This matrix captures the relationships between different features.
- **Calculate Eigenvalues and Eigenvectors:** These help identify the principal components.
- **Transform the Data:** Project the original data onto the principal components.



# Principal Component Analysis

**Pro**

- **Simple & fast.**
- **Easy to interpret variance explained.**
- **Useful preprocessing step.**

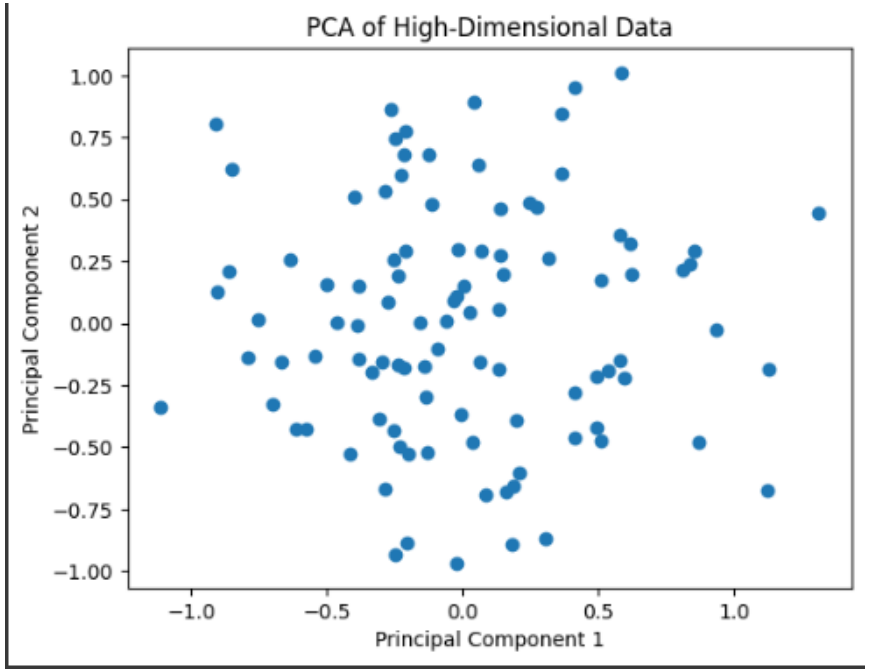
**Limitations**

- **Only linear relationships.**
- **Sensitive to scaling.**
- **May miss curved, non-linear structures.**
- **Components are not always interpretable.**

# Principal Component Analysis

**Example:**

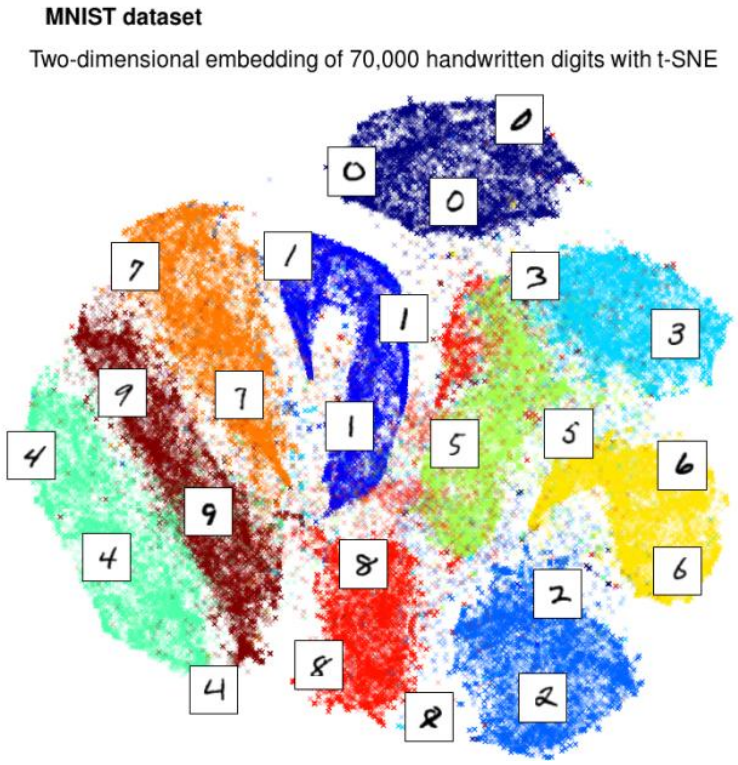
```
import numpy as np
from sklearn.decomposition import PCA
import matplotlib.pyplot as plt

# Generating a sample high-dimensional dataset
# For example, creating 100 samples with 50 features each
np.random.seed(42)
data = np.random.rand(100, 50)

# Applying PCA to reduce the dataset to 2 dimensions
pca = PCA(n_components=2)
transformed_data = pca.fit_transform(data)

# Plotting the transformed data
plt.scatter(transformed_data[:, 0], transformed_data[:, 1])
plt.title('PCA of High-Dimensional Data')
plt.xlabel('Principal Component 1')
plt.ylabel('Principal Component 2')
plt.show()
```



# t-Distributed Stochastic Neighbor Embedding

**T-SNE:** non-linear dimensionality reduction technique that helps visualize high-dimensional data. Its main goal is to preserve **local neighborhoods**: points close in high dimensions stay close in 2D.

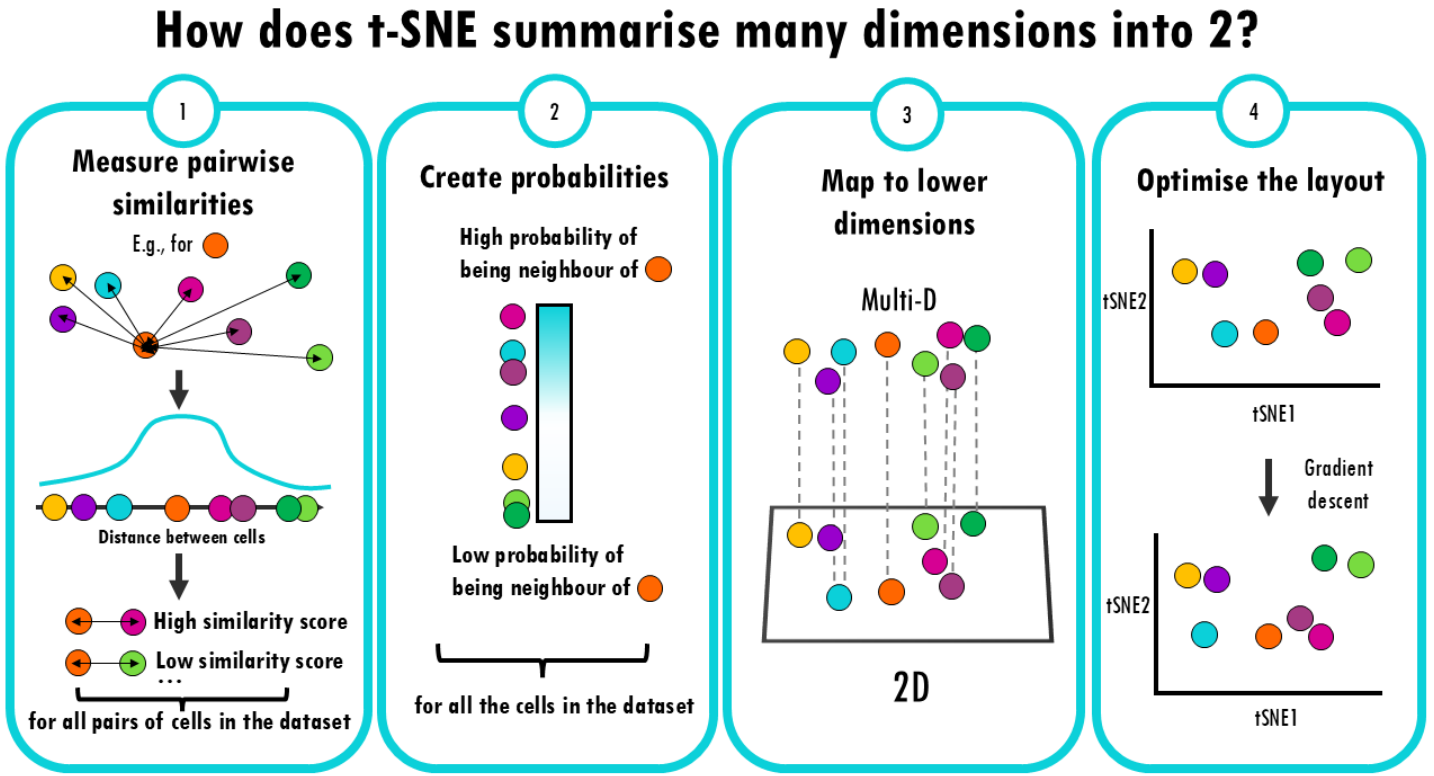
- Good for cluster visualization.
- Popular in NLP, genomics, images.



# t-Distributed Stochastic Neighbor Embedding

**How it works?**

- **Compute Similarities in High-Dim Space**
  - For each pair of points, compute a similarity using a **Gaussian kernel**.
  - Close points → **high probability** of being neighbors.
  - Far points → **low probability**.
- **Compute Similarities in Low-Dim Space**
  - Map points into 2D/3D (first guess is random and then adjusted)
  - Use a **Student t-distribution** to model neighbor probabilities.
- **Optimize with Gradient Descent**
  - Compare high- and low-dimensional similarities.
  - Minimize **Kullback–Leibler (KL) divergence**.
  - Adjust positions until embedding is stable.




*https://biostatsquid.com/*

# t-Distributed Stochastic Neighbor Embedding

**Pro**

- Captures non-linear patterns.
- Helpful in displaying local structures and clusters..

**Limitations**

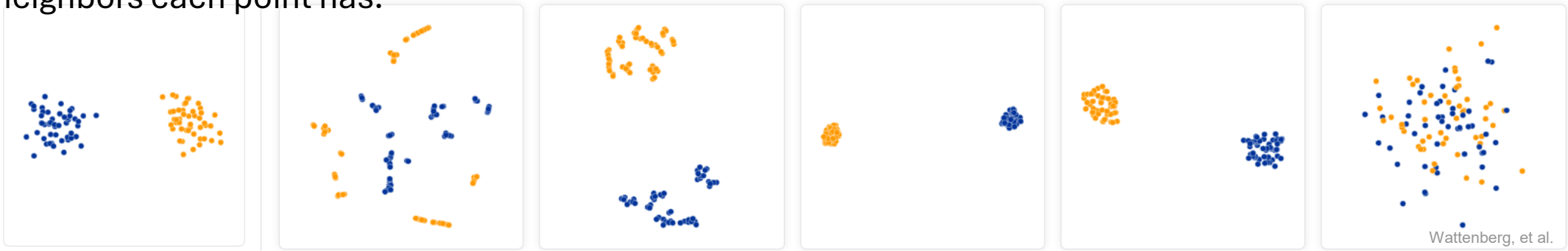
- **Computational cost:** t-SNE is computationally expensive, especially for large datasets.
- **Sensitivity to hyperparameters:** The performance of t-SNE is highly dependent on hyperparameters like perplexity and learning rate.
- **Lack of interpretability**: The resulting embeddings in t-SNE are often non-deterministic and lack a direct interpretable relationship with the original high-dimensional data.

# t-Distributed Stochastic Neighbor Embedding

**Hyperparameters**

- **Learning rate**: influences optimization.

- **Perplexity**: controls the balance between local and global structure. It is a guess about the number of close neighbors each point has.




*Original*

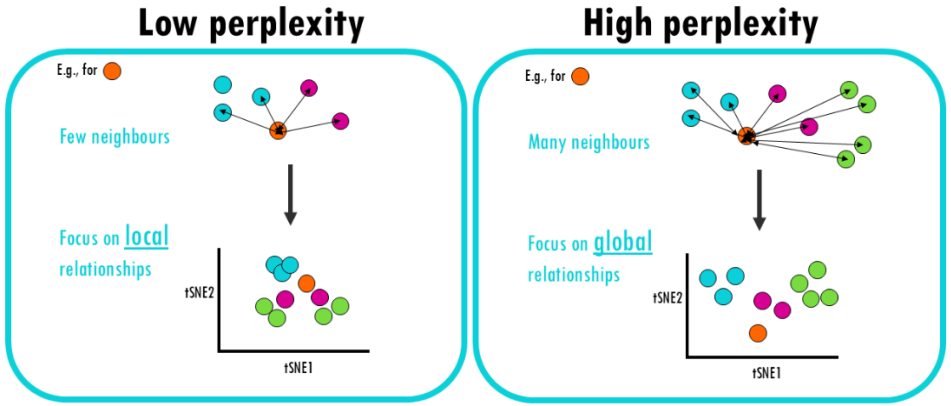
Perplexity: 2
Step: 5,000

Perplexity: 5
Step: 5,000

Perplexity: 30
Step: 5,000

Perplexity: 50
Step: 5,000

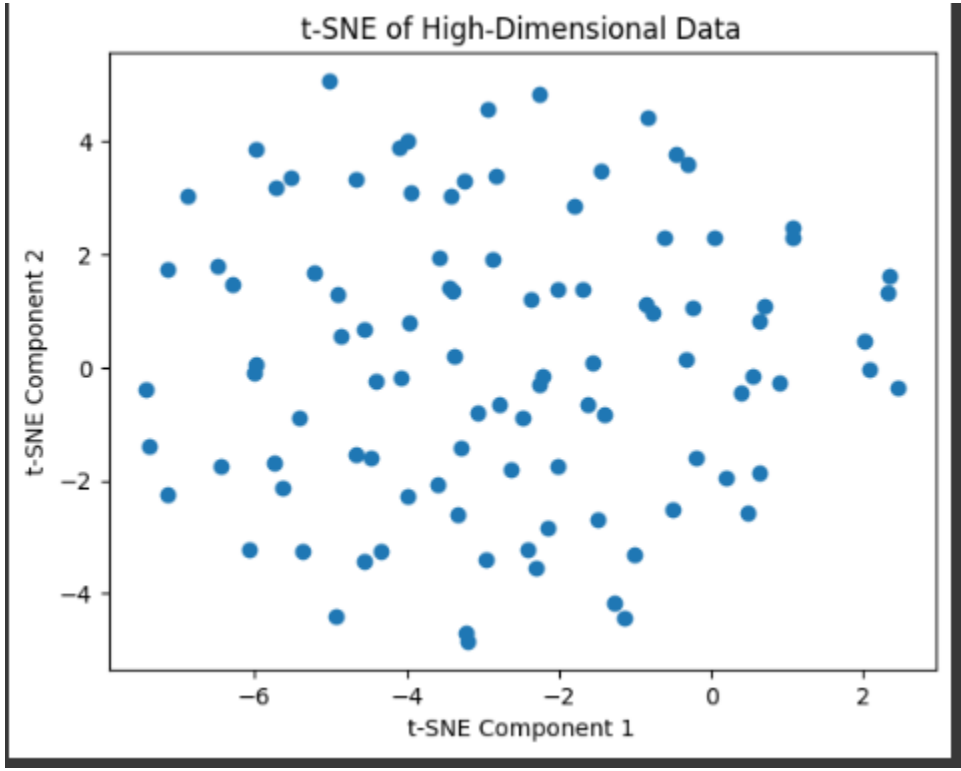
Perplexity: 100
Step: 5,000



# t-Distributed Stochastic Neighbor Embedding

**example:**

```
import numpy as np
from sklearn.manifold import TSNE
import matplotlib.pyplot as plt

# Generating a sample high-dimensional dataset
# For example, creating 100 samples with 50 features each
np.random.seed(42)
data = np.random.rand(100, 50)

# Applying t-SNE to reduce the dataset to 2 dimensions
tsne = TSNE(n_components=2, random_state=42)
tsne_results = tsne.fit_transform(data)

# Plotting the t-SNE results
plt.scatter(tsne_results[:, 0], tsne_results[:, 1])
plt.title('t-SNE of High-Dimensional Data')
plt.xlabel('t-SNE Component 1')
plt.ylabel('t-SNE Component 2')
plt.show()
```



# PCA vs t-SNE

| Characteristic | t-SNE | PCA |
| --- | --- | --- |
| Type | Non-linear dimensionality reduction | Linear dimensionality reduction |
| Goal | Preserve local pairwise similarities | Preserve global variance |
| Best used for | Visualizing complex, high-dimensional data | Data with linear structure |
| Output | Low-dimensional representation | Principal components |
| Use cases | Clustering, anomaly detection, NLP | Noise reduction, feature extraction |
| Computational intensity | High | Low |
| Interpretation | Harder to interpret | Easier to interpret |

# Uniform Manifold Approximation and Projection

**UMAP:** a relatively new technique for dimensionality reduction that is similar to t-SNE but often faster and better at preserving the global structure of the data. UMAP constructs a high-dimensional graph of the data and then optimizes a low-dimensional graph to be as structurally similar as possible

Ex:

- Single-cell RNA sequencing.
- Clear clusters of cell types.
- Scales to millions of cells.

**UMAP Concept**

High-dimensional data often lies on a lower-dimensional *manifold* (a kind of curved surface).
For example: a 3D object can look like a 2D surface when observed locally.
UMAP learns this underlying manifold structure.
Instead of reducing from 3D → 2D, UMAP reduces from **10,000 genes → 2D projections** for visualization.

# Uniform Manifold Approximation and Projection

**How it works?**

- **Construct High-Dimensional Graph**: Create a graph representing the high-dimensional data.

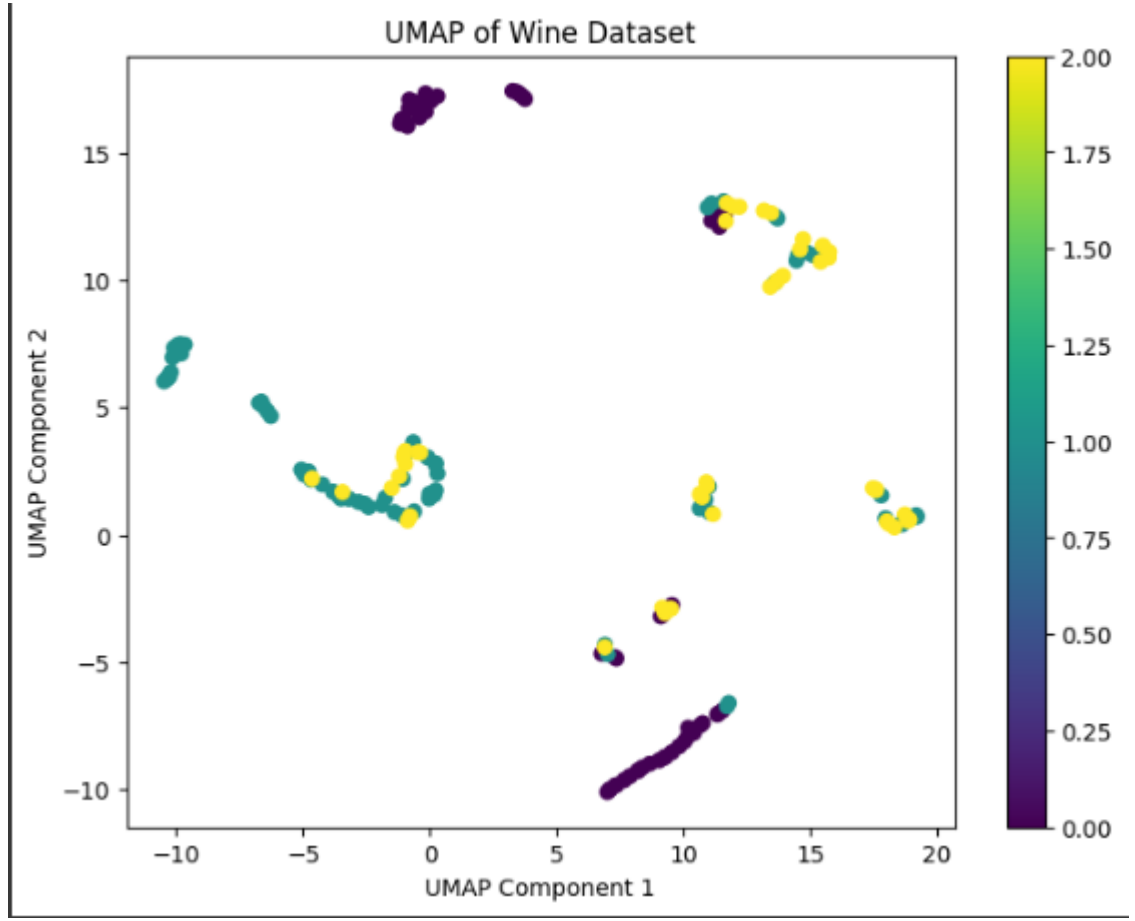
- **Optimize Low-Dimensional Graph**: Use optimization techniques to create a low-dimensional graph that maintains the structure of the high-dimensional graph, i.e., the topological structure

It relies on concepts from algebraic topology, such as simplicial complexes.
The details are complex, but the idea is to preserve the manifold structure.

# Uniform Manifold Approximation and Projection

**Example**

```
import umap
import matplotlib.pyplot as plt
from sklearn.datasets import load_wine

# Load dataset
data = load_wine()
X = data.data

# Apply UMAP
umap_model = umap.UMAP(n_neighbors=5, min_dist=0.3,
n_components=2)
X_umap = umap_model.fit_transform(X)

# Plotting
plt.figure(figsize=(8, 6))
plt.scatter(X_umap[:, 0], X_umap[:, 1], c=data.target,
cmap='viridis')
plt.xlabel('UMAP Component 1')
plt.ylabel('UMAP Component 2')
plt.title('UMAP of Wine Dataset')
plt.colorbar()
plt.show()
```



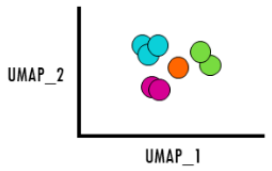
# Uniform Manifold Approximation and Projection

**Hyperparameters:**

n_neighbours is a hyperparameter which controls how much of the local vs global structure is preserved

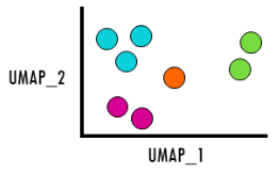
**Low n_neighbours**

Focus on local relationships




**High n_neighbours**

Focus on global relationships

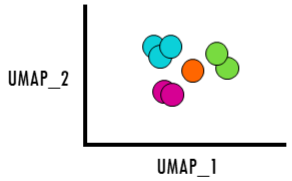



*https://biostatsquid.com/*

min_dist is the minimum distance between points in the low-dimensional space.

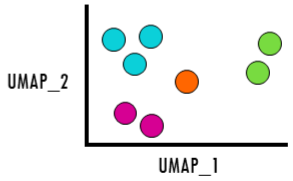
**Low min_dist**

Focus on local relationships

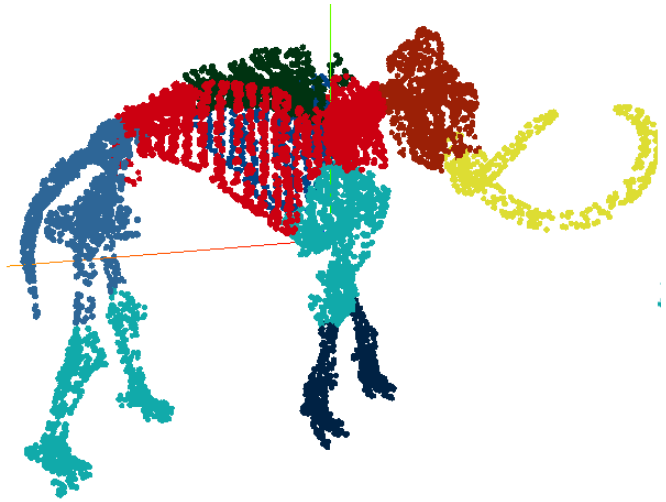


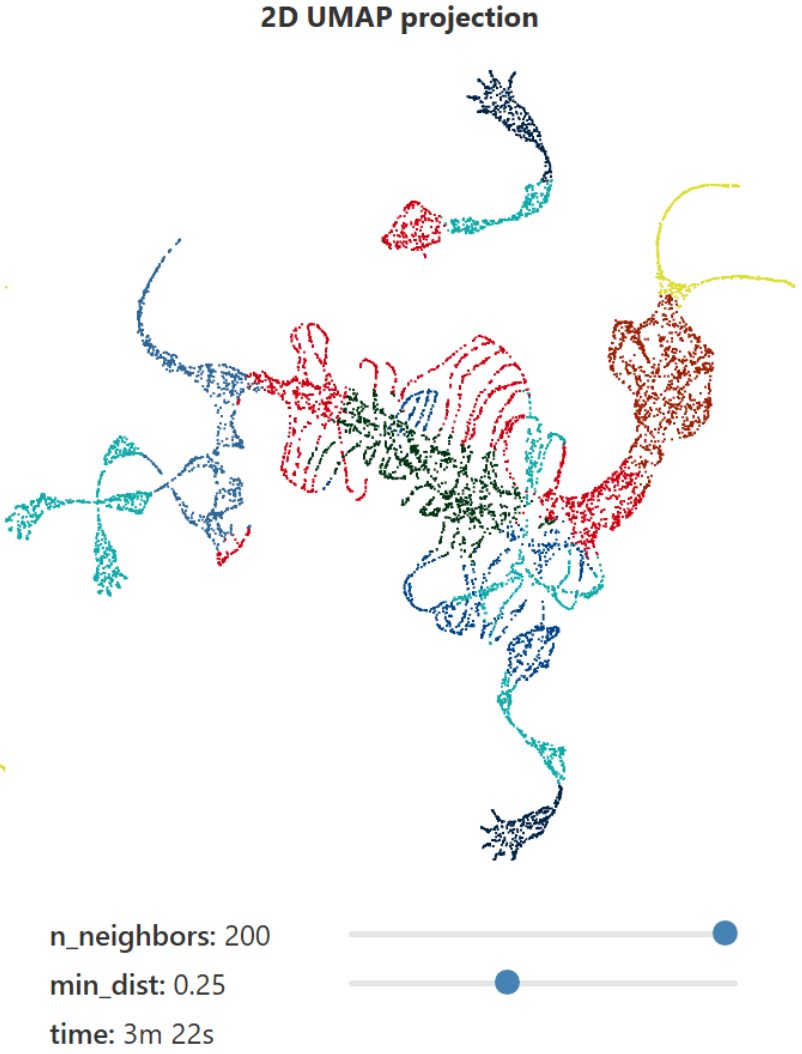

**High min_dist**

Focus on global relationships



# Uniform Manifold Approximation and Projection




Understanding UMAP

# PARALLEL COORDINATES

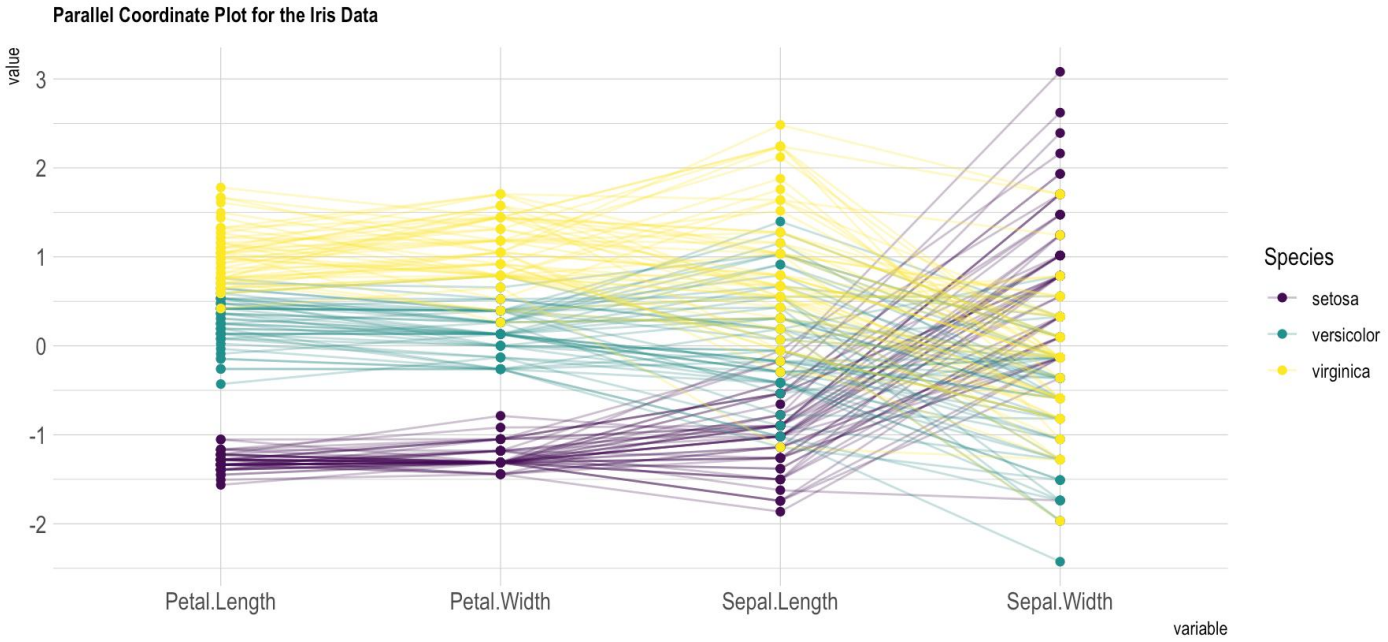
A common way of visualizing high-dimensional data. Each feature is represented as a vertical axis, and each data point is represented as a line that intersects each axis at the corresponding feature value.

**Pros**

No information loss from projection.
Direct comparison across variables.
Works well with interactivity.

**Cons**

Clutter with many lines/variables.
Hard to see global structures; ordering sensitivity.
Not ideal for huge datasets without sampling.




parallel coordinates show that species differ strongly on petal length and width

Parallel coordinates plot – from Data to Viz